KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

INCOME TAX ESTIMATION AND DETECTION SYSTEM

Submitted by

ATHISH M

25BCS011

Under the Guidance of

Dr.Boopalan Sundaramoorthy

Technical Head

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Income Tax Detection ”** submitted in partial fulfillment of the requirements for the course **Java** is a **record of my original work** carried out under the guidance of **Dr.Boopalan Sundaramoorthy** Technical Head,Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

ATHISH M

CERTIFICATE

This is to certify that the project entitled **"INCOME TAX DETECTION "** is a **bona fide work** carried out by **GOKUL S-2025-2026 [COMPUTER SCIENCE]**, **KG College of Arts&Science**, in partial fulfillment of the requirements for the course **[BSC.COMPUTER SCIENCE]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:10/04/2026

Submitted for the Viva-Voce Examination held on

10/04/26

Guide:

(Technical Head, KGCAS)

_____

_____

Head of the Department

(Technical Head, KGCAS)

TABLE OF CONTENTS

	S. No.	Topics	Page. No.
	1	Introduction	1

	1.1	Objective	**1**
	1.2	Overview	**1**
	1.3	Features	**2**
	2	**Program Analysis**	**3**
	2.1	Problem Definition	**3**
	2.2	Project Overview	**3**
	2.3	Scopes and Limitations	**3**
	3	**System Design**	**4**
	3.1	Modules	**4**
	4	**Methodology**	**5**
	4.1	Problem Analysis	**5**
	4.2	System Design	**5**
	4.3	Program Development	**6**
	4.4	Testing and Validation	**6**
	4.5	Result and Documentation	**6**
	4.6	Algorithm	**7**
	4.7	Flowchart	**7**
	5	**Source Code**	**8**
	6	**Input and Output Screens**	**14**
	7	**Discussion and Conclusion**	**15**
	7.1	Discussion	**15**
	7.2	Conclusion	**15**
	8	**References**	**16**

ABSTRACT

The Income Tax Estimation and Detection System GUI-based desktop application built using Java Swing. It automates the tax calculation process, eliminating manual errors and time delays. The system features a user-friendly interface where users can input their details to instantly compute their tax based on predefined income slabs. A core feature is its built-in fraud detection mechanism, which flags suspicious entries if the calculated tax is abnormally low compared to the declared income. It also securely saves taxpayer records to a text file

for future reference. This project effectively demonstrates object-oriented programming, Event Handling, and File I/O to deliver a reliable, automated tax assessment tool.

1

1. INTRODUCTION

Efficient tax calculation systems play a crucial role in managing financial operations. Traditionally, manual methods lead to errors and difficulty in maintaining proper records. The Income Tax Estimation and Detection System is a GUI-based Java application designed to automate the process of calculating tax and generating records. The application provides a user-friendly graphical interface to easily display the income, calculated tax, and any detected suspicious activity. This project is developed using Core Java and Java Swing concepts

1.1 Objective

The specific objectives of this project are:

To design and develop a GUI-based tax calculation system using Java Swing.

To automate the calculation of income taxes based on specific slabs.

To reduce manual errors in calculations.

To demonstrate the implementation of Core Java programming concepts (classes, objects, event handling, exception handling).

To provide fraud detection logic to flag suspicious tax entries.

1.2 Overview

The system is an event-driven GUI application developed using the Java. It helps simplify the tax calculation process.

2

1.3 Features

helps simplify the tax calculation process.

Graphical Interface: Provides a simple, user-friendly graphical window with buttons and text fields.

Automatic Calculation: Computes the total tax amount automatically based on conditional income slabs.

Fraud Detection: Checks if the income exceeds 1,000,000 but the calculated tax is suspiciously low.

File Handling: Saves taxpayer details securely to a textfile (taxdata.txt).

Cross-Platform Mobile Support: Expanding the current Desktop GUI into a Mobile Application (Android/iOS) to allow users to calculate and check their tax status on the go.

Advanced Fraud Detection (AI-Powered): Moving beyond simple conditional checks to Machine Learning models that can detect complex tax evasion patterns based on historical data trends.

Data Visualization Dashboard: Implementation of graphical representation tools like JFreeChart to provide taxpayers with visual insights through Bar Charts and Pie Charts for better financial analysis.

2. PROGRAM ANALYSIS

2.1 Problem Definition

Manual tax calculation creates challenges like errors, time delays, and lack of automated records. This makes it difficult to track past data and increases the risk of financial discrepancies. An automated digital system is needed to solve these issues effectively.

2.2 Project Overview

The system provides a GUI where users can interact using buttons and text fields. It automatically calculates the exact tax amount, detects fraudulent high-income anomalies, and securely saves taxpayer records to a local file for future reference

2.3 Scopes and Limitations

The system operates as a standalone desktop GUI application and uses flat-file storage rather than a relational database. Because of this, it is highly efficient for quick computations but currently lacks cloud support and complex database querying.

4

3. SYSTEM DESIGN

3.1 Modules

The main modules of the **INCOME TAX DETECTION** are explained below:

1. Data Entry Module:

Allows theuser to enter information. It uses Java Swing components like JTextField and handles exceptions for invalid data types.

2. Tax Calculation Module:

Uses conditional logic to evaluate the income against predefined brackets.

3. Fraud Detection Module:

Checks the tax against the stated income. Triggers a warning if income is>1,000,000 but tax is<10%.

4. Data Storage Module:

Uses FileWriter to save the name and income into a text document.

5. Exit Module:

Uses JTextArea and JOptionPane dialog boxes to display results and alerts.

5

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **INCOME TAX DETECTION** structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

• Problem Analysis
• System Design
• Program Developmentno
• Testing and Validation
• Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The main problems identified are manual calculation errors and the lack of automated records for transactions..

4.2. System Designn this stage, the overall structure of the system is designed before coding begins. The program flow and graphical user interface (GUI) structure are designed so that users can easily interact with the system using text fields and buttons.

4.3. Program Development

The program is implemented using the Java programming language. The development included the creation of base and derived classes (Person and TaxPayer), and the

implementation of a graphical user interface using Java Swing (JFrame, JPanel, JButton, JTextField). ActionListeners were used to handle button clicks. Core Java concepts such as classes, objects, exception handling, and methods were used during development

4.4. Testing and Validation

The program is implemented using the Java programming language. The development included the creation of base and derived classes (Person and TaxPayer), and the implementation of a graphical user interface using Java Swing (JFrame, JPanel, JButton, JTextField). ActionListeners were used to handle button clicks. Core Java concepts such as classes, objects, exception handling, and methods were used during development

4.5. Result and Documentation

After successful testing, the results of the system are documented. The system calculates the correct tax, displays alerts for suspicious income, and generates a file output.

7

4.6 Algorithm

1.Start the program and load the GUI window.

2.User enters Name and Income in the text fields

3.If the user clicks "Enter Details", validate and store the details.

4.If the user clicks "Calculate Tax", compute the total tax based on slabs and display it

5.If the user clicks "Detect Fraud", check the income and tax against fraud rules and display an alert.

6.If the user clicks "Save to File", write the details to a .txt file.

7.End the program when the window is closed.

4.7. Flowchart

Start

↓

Enter Details in GUI

↓

Click Button (Calculate / Detect / Save)

↓

Process Data (Calculate Tax / Check Fraud / File I/O)

↓Display Output in Text Area / Dialog Box

5.SOURCE CODE

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;

// Base class
class Person {
    protected String name;

    Person(String name) {
        this.name = name;
    }
}

// Derived class
class TaxPayer extends Person {
```

```
    private double income;

    TaxPayer(String name, double income) {
        super(name);
        this.income = income;
    }

    public double getIncome() {
        return income;
    }
```

9

```
    public void setIncome(double income) {
        this.income = income;
    }

    public String getName() {
        return name;
    }
}

// Tax C
class TaxCalculator {
    public double calculateTax(double income) {
        if (income < 250000)
            return 0;
        else if (income < 500000)
            return income * 0.05;
        else if (income < 1000000)
            return income * 0.1;
        else
            return income * 0.2;
    }

    public String detectFraud(double income, double tax) {
```

```
        if (income > 1000000 && tax < income * 0.1) {
            return "⚠ Suspicious Income Detected!";
        } else {
            return "No suspicious activity.";
        }
```

10

```
// GUI Main class
public class TaxSystemGUI extends JFrame {

    private JTextField nameField, incomeField;
    private JTextArea outputArea;
    private TaxPayer tp;
    private TaxCalculator calc;

    public TaxSystemGUI() {
        setTitle("Income Tax Estimation and Detection System");
        setSize(500, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        calc = new TaxCalculator();

        // Input Panel
        JPanel inputPanel = new JPanel(new GridLayout(2, 2, 5, 5));
        inputPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        inputPanel.add(new JLabel("Name:"));
        nameField = new JTextField();
        inputPanel.add(nameField);
        inputPanel.add(new JLabel("Income:"));
        incomeField = new JTextField();
        inputPanel.add(incomeField);
        add(inputPanel, BorderLayout.NORTH);
```

11

```
  JPanel buttonPanel = new JPanel(new GridLayout(1, 4, 5, 5));
    JButton enterBtn = new JButton("Enter Details");
    JButton calcBtn = new JButton("Calculate Tax");
    JButton detectBtn = new JButton("Detect Fraud");
    JButton saveBtn = new JButton("Save to File");
    buttonPanel.add(enterBtn);
    buttonPanel.add(calcBtn);
    buttonPanel.add(detectBtn);
    buttonPanel.add(saveBtn);
    add(buttonPanel, BorderLayout.CENTER);

    // Output Area
    outputArea = new JTextArea();
    outputArea.setEditable(false);
    JScrollPane scrollPane = new JScrollPane(outputArea);
    scrollPane.setBorder(BorderFactory.createTitledBorder("Output"));
    add(scrollPane, BorderLayout.SOUTH);

    // Button Actions
    enterBtn.addActionListener(e -> enterDetails());
    calcBtn.addActionListener(e -> calculateTax());
    detectBtn.addActionListener(e -> detectFraud());
    saveBtn.addActionListener(e -> saveToFile());
    setVisible(true);
}

private void enterDetails()
```

12

```
String name = nameField.getText();
        double income = Double.parseDouble(incomeField.getText());
        tp = new TaxPayer(name, income);
```

```
            outputArea.setText("Details Entered!\nName: " + name + "\nIncome: " +
income);
        } catch (NumberFormatException ex) {
             JOptionPane.showMessageDialog(this, "Enter valid income!", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }
    private void calculateTax() {
        if (tp == null) {
            JOptionPane.showMessageDialog(this, "Enter details first!", "Warning",
JOptionPane.WARNING_MESSAGE);
        } else {
            double tax = calc.calculateTax(tp.getIncome());
            outputArea.append("\nCalculated Tax: " + tax);
        }
    }
    private void detectFraud() {
        if (tp == null) {
            JOptionPane.showMessageDialog(this, "Enter details first!", "Warning",
JOptionPane.WARNING_MESSAGE);
        } else {
            double tax = calc.calculateTax(tp.getIncome());
            String result = calc.detectFraud(tp.getIncome(), tax);
             outputArea.append("\n" + result);
```

13

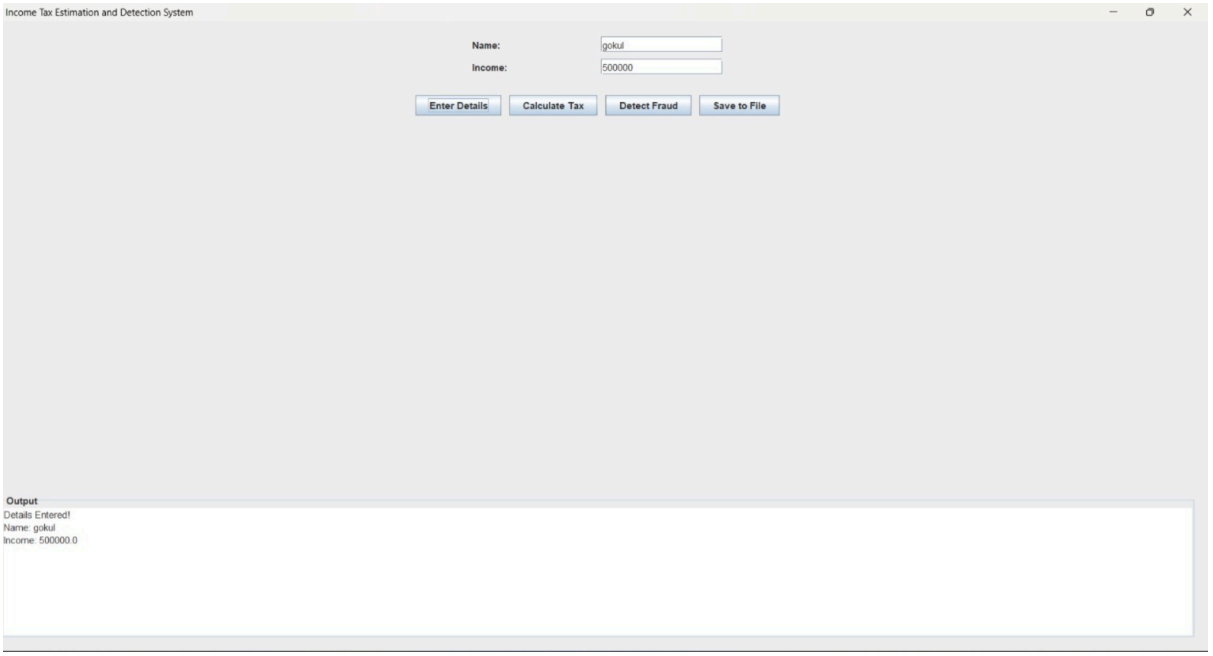
```
        }
    }

    private void saveToFile() {
        if (tp == null) {
            JOptionPane.showMessageDialog(this, "Enter details first!", "Warning",
JOptionPane.WARNING_MESSAGE);
        } else {
            try {
                FileWriter fw = new FileWriter("taxdata.txt", true);
```

```
                fw.write("Name: " + tp.getName() + ", Income: " + tp.getIncome() +
"\n");
                fw.close();
                outputArea.append("\nSaved to file!");
            } catch (IOException e) {
                        JOptionPane.showMessageDialog(this, "File error!", "Error",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(TaxSystemGUI::new);
    }
}
```


6.INPUT AND OUTPUT SCREEN



7.DISCUSSION AND CONCLUSION

7.1 Discussion

The system allows users to enter data and calculate taxes automatically using a graphical user interface. During the development process, Core Java programming concepts such as conditional statements, file handling, and Java Swing Event Handling were applied. The system successfully performs calculations and generates output interactively. The results show that the system reduces manual effort.

7.2 Conclusion

The project successfully demonstrates the practical application of Core Java and GUI programming concepts in developing a real-world desktop application. The system automates the process and reduces calculation errors. This project helps students understand how Java programming can be used to develop practical software systems. The project can be further enhanced by adding features such as database connectivity (MySQL) to manage taxpayer records more efficientl

8.REFERENCES

The following resources were used during the development of this project:

Books

Herbert Schildt - Java: The Complete Reference

Websites

https://www.oracle.com/java

https://www.geeksforgeeks.org

https://www.javatpoint.com